Atelier 1 SQL Complet

Requêtes, Calculs, Optimisation et Sécurité

Gestion commerciale : clients, commandes, produits, paiements

1. Connexion au serveur MySQL

Connexion au serveur MySQL distant hébergé sur Aiven avec SSL activé.

```
C:\Program Files\MySQL\MySQL Server 8.4\bin>mysql -h mysql-349bf5e7-atelier6.g.aivencloud.com -P 19213 -u avnadmin -p --ssl-mode=REQUIRED
Enter password: ************************
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 492361
Server version: 8.4.11 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

Connexion réussie au serveur MySQL 8.4.11.

2. Création de la base et des tables

Création de la base atelier_commerce et des cinq tables avec leurs clés primaires et étrangères.

```
mysql> drop database atelier_commerce;
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE DATABASE atelier_commerce;
Query OK, 1 row affected (0.05 sec)

mysql> USE atelier_commerce;
Database changed
mysql> CREATE TABLE clients (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom VARCHAR(50),
    ->   ville VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

Création de la base et de la table clients.

```
mysql>
mysql> CREATE TABLE produits (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   nom_produit VARCHAR(50),
    ->   prix_unitaire DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE commandes (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   client_id INT,
    ->   date_commande DATE,
    ->   FOREIGN KEY (client_id) REFERENCES clients(id)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE details_commandes (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   commande_id INT,
    ->   produit_id INT,
    ->   quantite INT,
    ->   FOREIGN KEY (commande_id) REFERENCES commandes(id),
    ->   FOREIGN KEY (produit_id) REFERENCES produits(id)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> CREATE TABLE paiements (
    ->   id INT AUTO_INCREMENT PRIMARY KEY,
    ->   commande_id INT,
    ->   montant DECIMAL(10,2),
    ->   mode_paiement VARCHAR(30),
    ->   date_paiement DATE,
    ->   FOREIGN KEY (commande_id) REFERENCES commandes(id)
    -> );
Query OK, 0 rows affected (0.10 sec)
```

Création des tables produits, commandes, details_commandes et paiements.

3. Insertion des données

Insertion des données d'exemple dans chaque table.

```
mysql> INSERT INTO clients (nom, ville)
    -> VALUES ('Alice','Paris'), ('Bob','Lyon'), ('Charlie','Toulouse'), ('Dina','Nice');
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO produits (nom_produit, prix_unitaire)
    -> VALUES ('Ordinateur',950), ('Souris',20), ('Clavier',45), ('Écran',250);
Query OK, 4 rows affected (0.05 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO commandes (client_id, date_commande)
    -> VALUES (1,'2025-01-10'), (1,'2025-01-20'), (2,'2025-02-01'), (3,'2025-02-10');
Query OK, 4 rows affected (0.06 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO details_commandes (commande_id, produit_id, quantite)
    -> VALUES (1,1,1), (1,2,2), (2,3,1), (3,4,2), (4,2,3);
Query OK, 5 rows affected (0.06 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO paiements (commande_id, montant, mode_paiement, date_paiement)
    -> VALUES (1,990,'Carte','2025-01-12'),
    ->        (2,45,'Espèces','2025-01-21'),
    ->        (3,500,'Virement','2025-02-05');
Query OK, 3 rows affected (0.05 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Insertion des clients, produits, commandes, détails et paiements.

4. Jointures

Interrogation de plusieurs tables à l'aide des différents types de jointures.

```
mysql> SELECT c.nom, cmd.id AS commande, p.nom_produit, dc.quantite
    -> FROM clients c
    -> INNER JOIN commandes cmd ON c.id = cmd.client_id
    -> INNER JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> INNER JOIN produits p ON dc.produit_id = p.id;
+---------+----------+-------------+----------+
| nom     | commande | nom_produit | quantite |
+---------+----------+-------------+----------+
| Alice   |        1 | Ordinateur  |        1 |
| Alice   |        1 | Souris      |        2 |
| Alice   |        2 | Clavier     |        1 |
| Bob     |        3 | Écran       |        2 |
| Charlie |        4 | Souris      |        3 |
```

INNER JOIN : clients ayant commandé, avec les produits.

```
mysql> SELECT c.nom, cmd.id AS num_commande
    -> FROM clients c
    -> LEFT JOIN commandes cmd ON c.id = cmd.client_id;
+---------+--------------+
| nom     | num_commande |
+---------+--------------+
| Alice   |            1 |
| Alice   |            2 |
| Bob     |            3 |
| Charlie |            4 |
| Dina    |         NULL |
+---------+--------------+
5 rows in set (0.05 sec)
```

LEFT JOIN : tous les clients, y compris Dina sans commande.

```
mysql> SELECT c.nom, cmd.id, cmd.date_commande
    -> FROM clients c
    -> RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
+---------+----+---------------+
| nom     | id | date_commande |
+---------+----+---------------+
| Alice   |  1 | 2025-01-10    |
| Alice   |  2 | 2025-01-20    |
| Bob     |  3 | 2025-02-01    |
| Charlie |  4 | 2025-02-10    |
+---------+----+---------------+
4 rows in set (0.05 sec)
```

RIGHT JOIN : toutes les commandes avec leur client.

```
mysql> SELECT c.nom, cmd.id AS num_commande
    -> FROM clients c LEFT JOIN commandes cmd ON c.id = cmd.client_id
    -> UNION
    -> SELECT c.nom, cmd.id AS num_commande
    -> FROM clients c RIGHT JOIN commandes cmd ON c.id = cmd.client_id;
+---------+--------------+
| nom     | num_commande |
+---------+--------------+
| Alice   |            1 |
| Alice   |            2 |
| Bob     |            3 |
| Charlie |            4 |
| Dina    |         NULL |
+---------+--------------+
5 rows in set (0.05 sec)
```

FULL JOIN simulé avec UNION.

5. Agrégations et calculs

Calcul de totaux et de moyennes avec SUM, AVG, GROUP BY et HAVING.

```
mysql> SELECT cmd.id AS commande, SUM(p.prix_unitaire * dc.quantite) AS total_commande
    -> FROM commandes cmd
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY cmd.id;
+----------+----------------+
| commande | total_commande |
+----------+----------------+
|        1 |         990.00 |
|        2 |          45.00 |
|        3 |         500.00 |
|        4 |          60.00 |
+----------+----------------+
4 rows in set (0.17 sec)
```

Montant total par commande.

```
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total_client
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom
    -> HAVING total_client > 500;
+-------+--------------+
| nom   | total_client |
+-------+--------------+
| Alice |      1035.00 |
+-------+--------------+
1 row in set (0.07 sec)
```

Total par client supérieur à 500.

```
mysql> SELECT p.nom_produit, AVG(p.prix_unitaire) AS prix_moyen
    -> FROM produits p
    -> GROUP BY p.nom_produit;
+-------------+------------+
| nom_produit | prix_moyen |
+-------------+------------+
| Ordinateur  | 950.000000 |
| Souris      |  20.000000 |
| Clavier     |  45.000000 |
| Écran       | 250.000000 |
+-------------+------------+
4 rows in set (0.05 sec)
```

Prix moyen par produit.

6. Fonctions SQL

Classement, calcul de délais et limitation des résultats avec IF(), DATEDIFF() et LIMIT.

```
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total,
    ->        IF(SUM(p.prix_unitaire * dc.quantite) > 800, 'Premium', 'Standard') AS categorie
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom;
+---------+---------+-----------+
| nom     | total   | categorie |
+---------+---------+-----------+
| Alice   | 1035.00 | Premium   |
| Bob     |  500.00 | Standard  |
| Charlie |   60.00 | Standard  |
+---------+---------+-----------+
3 rows in set (0.09 sec)
```

IF() : classement Premium / Standard.

```
mysql> SELECT p.id AS paiement, cmd.id AS commande,
    ->        DATEDIFF(p.date_paiement, cmd.date_commande) AS delai_jours
    -> FROM paiements p
    -> JOIN commandes cmd ON p.commande_id = cmd.id;
+----------+----------+-------------+
| paiement | commande | delai_jours |
+----------+----------+-------------+
|        1 |        1 |           2 |
|        2 |        2 |           1 |
|        3 |        3 |           4 |
+----------+----------+-------------+
3 rows in set (0.07 sec)
```

DATEDIFF() : délai entre commande et paiement.

```
mysql> SELECT c.nom, SUM(p.prix_unitaire * dc.quantite) AS total
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> GROUP BY c.nom
    -> ORDER BY total DESC
    -> LIMIT 2;
+-------+---------+
| nom   | total   |
+-------+---------+
| Alice | 1035.00 |
| Bob   |  500.00 |
+-------+---------+
2 rows in set (0.09 sec)
```

LIMIT : les deux meilleurs clients.

7. Sous-requêtes IN / NOT IN / EXISTS

Filtrage des clients selon leurs commandes et leurs paiements.

```
mysql> SELECT nom FROM clients WHERE id IN (SELECT client_id FROM commandes);
+---------+
| nom     |
+---------+
| Alice   |
| Bob     |
| Charlie |
+---------+
3 rows in set (0.10 sec)
```

IN : clients ayant des commandes.

```
mysql> SELECT nom FROM clients WHERE id NOT IN (SELECT client_id FROM commandes);
+------+
| nom  |
+------+
| Dina |
+------+
1 row in set (0.08 sec)
```

NOT IN : clients sans commande.

```
mysql> SELECT nom FROM clients
    -> WHERE id IN (
    ->   SELECT client_id FROM commandes
    ->   WHERE id IN (SELECT commande_id FROM paiements)
    -> );
+-------+
| nom   |
+-------+
| Alice |
| Bob   |
+-------+
2 rows in set (0.05 sec)
```

IN imbriqué : clients dont une commande est payée.

```
mysql> SELECT nom FROM clients c
    -> WHERE EXISTS (
    ->   SELECT 1 FROM commandes cmd
    ->   JOIN paiements p ON cmd.id = p.commande_id
    ->   WHERE cmd.client_id = c.id
    -> );
+-------+
| nom   |
+-------+
| Alice |
| Bob   |
+-------+
2 rows in set (0.05 sec)
```

EXISTS : clients ayant effectué un paiement.

8. Optimisation : index et EXPLAIN

Création d'index et vérification de leur utilisation : la colonne key affiche idx_nom_produit.

```
mysql> CREATE INDEX idx_nom_produit ON produits(nom_produit);
Query OK, 0 rows affected (0.52 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> CREATE INDEX idx_client_ville ON clients(ville);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> EXPLAIN SELECT * FROM produits WHERE nom_produit = 'Souris';
+----+-------------+----------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
| id | select_type | table    | partitions | type | possible_keys   | key             | key_len | ref   | rows | filtered | Extra |
+----+-------------+----------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
|  1 | SIMPLE      | produits | NULL       | ref  | idx_nom_produit | idx_nom_produit | 203     | const |    1 |   100.00 | NULL  |
+----+-------------+----------+------------+------+-----------------+-----------------+---------+-------+------+----------+-------+
1 row in set, 1 warning (0.17 sec)
```

Création des index et résultat de EXPLAIN.

9. Vue

Création de la vue vue_ventes et lecture avec filtre sur le montant.

```
mysql> CREATE VIEW vue_ventes AS
    -> SELECT c.nom, p.nom_produit, dc.quantite,
    ->        (p.prix_unitaire * dc.quantite) AS montant,
    ->        pa.mode_paiement
    -> FROM clients c
    -> JOIN commandes cmd ON c.id = cmd.client_id
    -> JOIN details_commandes dc ON cmd.id = dc.commande_id
    -> JOIN produits p ON dc.produit_id = p.id
    -> LEFT JOIN paiements pa ON cmd.id = pa.commande_id;
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> SELECT * FROM vue_ventes WHERE montant > 200;
+-------+-------------+----------+---------+---------------+
| nom   | nom_produit | quantite | montant | mode_paiement |
+-------+-------------+----------+---------+---------------+
| Alice | Ordinateur  |        1 |  950.00 | Carte         |
| Bob   | Écran       |        2 |  500.00 | Virement      |
+-------+-------------+----------+---------+---------------+
2 rows in set (0.05 sec)
```

*Création de la vue et SELECT * FROM vue_ventes.*

10. Gestion des droits (DCL)

Création des utilisateurs, attribution et retrait de droits avec GRANT, SHOW GRANTS et REVOKE.

```
mysql> CREATE USER 'vendeur'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0.26 sec)

mysql> CREATE USER 'manager'@'localhost' IDENTIFIED BY 'admin';
Query OK, 0 rows affected (0.05 sec)
```

Création des utilisateurs vendeur et manager.

```
mysql>
mysql> GRANT SELECT, INSERT, UPDATE ON atelier_commerce.* TO 'vendeur'@'localhost';
Query OK, 0 rows affected, 1 warning (0.05 sec)

mysql> GRANT ALL PRIVILEGES ON atelier_commerce.* TO 'manager'@'localhost';
Query OK, 0 rows affected, 1 warning (0.05 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.10 sec)

mysql>
mysql> SHOW GRANTS FOR 'manager'@'localhost';
+-----------------------------------------------------------------------+
| Grants for manager@localhost                                          |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO "manager"@"localhost"                           |
| GRANT ALL PRIVILEGES ON "atelier_commerce".* TO "manager"@"localhost" |
+-----------------------------------------------------------------------+
2 rows in set (0.04 sec)

mysql>
mysql> REVOKE UPDATE ON atelier_commerce.* FROM 'vendeur'@'localhost';
Query OK, 0 rows affected, 1 warning (0.05 sec)
```

GRANT, FLUSH PRIVILEGES, SHOW GRANTS et REVOKE.